EECS 348 Assignment 4 – GenAI Analysis

Recursive Object-Oriented Sudoku Solver | Author: Justin Whitaker | Date: 10/15/2026

a. Names of the Two GenAI Programs

1. **GenAI #1 – Claude Opus** (Anthropic)
2. **GenAI #2 – Gemini, Flash model** (Google)

(ChatGPT and Copilot were not used, so the rule about using only one of those two is satisfied.)

b. How Each GenAI Was Accessed

GenAI	Access method
Claude Opus	The agent chat panel of the **Google Antigravity IDE** installed on my Windows computer (https://antigravity.google), with the model selector set to Claude Opus. The prompt was typed into a new chat.
Gemini (Flash)	Also through the **Google Antigravity IDE** on my computer, in a separate session with the model set to Gemini Flash. That session was told not to read any project files or use any tools, so it only had the prompt below to work from.

c. Prompt Used (identical for both GenAIs)

> `Write a C++ program using objects (classes), not standalone functions, that solves Sudoku puzzles using a recursive depth-first search with backtracking. The program must read five puzzle files (puzzle1.txt through puzzle5.txt). Each file contains a 9x9 grid where digits are separated by spaces and an underscore (_) indicates a blank cell. For each file, print the puzzle file name, the original puzzle, and every solution to the puzzle (there may be more than one), or "No solution found" if no solution exists. For each empty cell, compute the list of candidate digits not already used in that cell's row, column, or 3x3 block; if there are no candidates, backtrack; otherwise try each candidate recursively until all 81 cells are filled.`

d. Generated Code

The full, unedited code from both GenAIs is in **Appendix A** (Claude Opus) and **Appendix B** (Gemini) at the end of this document.

- **Claude Opus** made three classes: `SudokuBoard` (stores the grid, loads files, checks digits), `SudokuSolver` (recursion that saves every solution in a `std::vector`), and `PuzzleRunner` (loops over the files and prints).
- **Gemini** made one class, `SudokuSolver`, that does everything: it loads the file, finds candidates, runs the recursion, and prints each solution as soon as it is found.

e. Analysis: Correctness, Execution Time, Space Complexity, Maintainability

How I tested: I compiled both programs with `g++ -std=c++17 -Wall -Wextra` (no optimization, like a normal cycle-server build) on Ubuntu Linux and ran them on the five assignment puzzles. Then I ran some edge-case files and a timing test on a very hard puzzle (Arto Inkala's "world's hardest Sudoku", copied into all five files). Times are the best of 3 runs.

e.1 Correctness

Test	Claude Opus	Gemini
puzzle1–5.txt (the assignment files)	Correct. puzzle1 matches solution1.txt; puzzle2 and puzzle3 have 1 solution each; puzzle4 prints "No solution found"; puzzle5 has **2** solutions.	Correct. The grids are exactly the same as Claude's (checked with `diff`).
Clues that break the rules (a filled grid with two 5s in row 1)	**Wrong:** prints it as "Solution 1" even though it is not a valid Sudoku.	**Wrong:** same mistake, prints the invalid grid as "Solution 1".
Invalid character (x in place of _)	Rejects the file, but the message is vague ("Could not read puzzle2.txt").	**Wrong:** turns `'x' - '0'` into the number **72**, prints "72" in the grid, and calls that a solution. A token like "12" would quietly be read as 1.
File too short / extra data after 81 cells	Catches files that are too short; ignores extra data.	Catches files that are too short; ignores extra data.
Reports how many solutions were found	Yes ("Solutions found: N")	No (the user has to count them)

Both programs are correct for the five required puzzles. Neither one checks that the starting clues are legal, so a bad puzzle file can produce a wrong "solution". Gemini's input handling is the weaker of the two.

e.2 Execution Time

Both use the same approach (depth-first search with backtracking). The worst case is exponential, $\mathbf{O(9^k)}$ for *k* empty cells. What makes them different is how much work each recursive call does:

Work per recursive call	Claude Opus	Gemini
Finding the next empty cell	Starts at cell 0 every time: up to 81 checks	Starts at cell 0 every time: up to 81 checks
Building the candidate list	Calls `canPlace` for each of the 9 digits, and each call scans 27 cells: about **243 checks**	Scans the row, column, and block once (27 checks), then loops over 9 digits: about **36 checks**
Heap allocations	1 (`std::vector<int>` of candidates)	2 (`std::vector<bool>(10)` and `std::vector<int>`)
Saving a solution	Copies the whole board into a vector	Prints it right away

Measured run time	Claude Opus	Gemini

5 assignment puzzles	0.01–0.02 s	0.02 s
5 × "world's hardest" puzzle	**11.91 s**	15.25 s

Gemini does far fewer comparisons, but it still ran **about 28% slower**. Its two heap allocations per call, plus the slow bit access of `std::vector<bool>`, cost more than Claude's extra comparisons. Both programs also waste time rescanning from cell 0 on every call. Either one is fast enough for the assignment puzzles, but both are slow on hard puzzles.

e.3 Space Complexity

- **Recursion stack:** both programs go at most $k \le 81$ calls deep, so the stack is **O(k)**. Each stack frame also owns a heap-allocated candidate vector (Claude: 1 per frame, Gemini: 2 per frame while it is building the list).
- **Storing solutions: Claude stores every solution** (`std::vector<SudokuBoard>`, about 324 bytes each), so its memory is **O(S)** for S solutions. Gemini prints each solution and throws it away, so it uses **O(1)** extra memory for solutions. This barely matters for these puzzles ($S \le 2$), but a puzzle with few clues can have millions of solutions, and then Claude's program would run out of memory.
- **Measured peak memory** on the assignment puzzles: about 4.1 MB for both (mostly the C++ runtime).

e.4 Maintainability

Aspect	Claude Opus	Gemini
Object-oriented design	**3 classes, each with one job (Board / Solver / Runner). The solver can be reused or tested without file I/O.**	One large class that mixes file reading, solving, printing, and a hard-coded file list.
Constants / magic numbers	Uses `SIZE` and `BOX` constants, but still has 9s written directly in some loops.	Uses `GRID_SIZE` and `SUBGRID_SIZE`, but also writes `1`, `9`, and `10` directly.
Comments	Sparse: one line per function.	**Better: one comment per function plus comments inside the functions.**
Readability	`findEmpty` uses its reference parameters as the loop counters, which is clever but hard to follow.	Easy to read and very direct.
Flexibility	The file list is passed to the constructor, so it is easy to change.	The file list is hard-coded inside a member function.
Portability	Uses a C++11 brace-initializer list in `main`.	Plain C++11.

f. Selection and Justification

I chose the Claude Opus program as my starting point. Reasons:

1. **Correctness:** both pass all five puzzles, but Claude's program already rejects bad characters. Gemini read x as 72 and printed a wrong answer. Claude also prints the "Solutions found" count.
2. **Execution time:** Claude ran **22% faster** on the hard puzzle test (11.91 s vs. 15.25 s). Its slowest part (scanning 27 cells for each digit) is easy to fix with lookup tables.

3. **Maintainability / OO design:** the assignment asks for an object-oriented program. Claude's three single-purpose classes fit that better than Gemini's one large class, and they are easier to change.
4. **Space:** Claude's one weakness, storing every solution, is a small change to fix. I took Gemini's idea of printing solutions as they are found.

g. Improvements Made to the Selected Code

Metric	What I changed	Result
Correctness	• While loading, each clue is checked with `canPlace`. If a clue breaks a rule, `givensValid` is set to false and the solver prints "No solution found" without searching. • The loader now rejects files with *more* than 81 entries, as well as files with fewer. • Error messages now say what went wrong ("Could not open ...", "Invalid entry 'x' in ...", "has fewer/more than 81 cells").	Same correct output for puzzle1–5 (puzzle5 still shows both solutions). The rule-breaking grid now prints "No solution found" instead of a wrong solution, and every malformed file gets a specific error message.
Execution time	• Added `rowUsed`, `colUsed`, and `boxUsed` lookup tables that are updated by `place()` and `removeDigit()`. `canPlace` is now **O(1)** (3 lookups instead of a 27-cell scan), so building the candidate list drops from about 243 checks to 27. • `search(cellIndex)` continues from the next cell instead of rescanning from cell 0. • Candidates go in a fixed `int[9]` array, so there are **no heap allocations** inside the recursion.	5 × hardest puzzle: **11.91 s → 1.14 s (about 10.4× faster)**, and 13× faster than Gemini. The assignment puzzles run in about 0.01 s.
Space complexity	• Solutions are printed as soon as they are found (through an `std::ostream&` passed to the solver) instead of being saved in a `std::vector`. • The per-frame candidate vector became a 36-byte stack array.	Solution storage goes from **O(S) to O(1)**. Total extra memory is O(k) stack frames ($k \le 81$) plus 270 bytes of lookup tables, no matter how many solutions there are.
Maintainability	• Kept the three-class design, and added named constants `EMPTY` and `CELL_COUNT` to replace raw numbers. • Replaced the clever `findEmpty` with a clear skip loop in `search`. • Added prologue comments, a summary comment for each block, and comments on every line. Each block says whether the code came from Claude, was inspired by Gemini, or was written by me.	Code that is easier to read, fully commented, credits its sources, and builds without warnings on older and newer g++ versions.

	• Replaced the brace-initializer list in `main` with `push_back` calls, and checked that the code compiles warning-free with `-std=c++11 -Wall -Wextra -pedantic` so it builds on older cycle-server compilers.	

Summary of measurements

Program	Assignment puzzles correct?	Rejects bad clues / input?	5 × hardest puzzle	Solution storage
Gemini (raw)	Yes	No / No	15.25 s	O(1)
Claude Opus (raw)	Yes	No / Partly	11.91 s	O(S)
Final Assignment4.cpp	**Yes**	**Yes / Yes**	**1.14 s**	**O(1)**

Appendix A – Raw Code Generated by Claude Opus

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <array>

// Represents a 9x9 Sudoku grid. 0 means an empty cell.
class SudokuBoard {
public:
    static const int SIZE = 9;
    static const int BOX = 3;

    SudokuBoard() {
        for (auto& row : cells) row.fill(0);
    }

    // Load a board from a file. Digits and '_' are separated by whitespace.
    bool loadFromFile(const std::string& filename) {
        std::ifstream in(filename);
        if (!in) {
            return false;
        }
        std::string token;
        int count = 0;
        while (count < SIZE * SIZE && in >> token) {
            int r = count / SIZE;
            int c = count % SIZE;
            if (token == "_") {
                cells[r][c] = 0;
            } else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') {
                cells[r][c] = token[0] - '0';
            } else {
                return false; // invalid token
            }
            ++count;
        }
        return count == SIZE * SIZE;
    }

    int get(int r, int c) const { return cells[r][c]; }
    void set(int r, int c, int v) { cells[r][c] = v; }

    // Returns true if value v can be placed at (r, c) without breaking the rules.
    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < SIZE; ++i) {
            if (cells[r][i] == v) return false;
            if (cells[i][c] == v) return false;
        }
        int br = (r / BOX) * BOX;
        int bc = (c / BOX) * BOX;
        for (int i = br; i < br + BOX; ++i)
            for (int j = bc; j < bc + BOX; ++j)
                if (cells[i][j] == v) return false;
        return true;
    }

    // Returns the list of digits that could legally go into (r, c).
    std::vector<int> candidates(int r, int c) const {
        std::vector<int> result;
        for (int v = 1; v <= 9; ++v) {
            if (canPlace(r, c, v)) result.push_back(v);
        }
        return result;
    }

    // Finds the first empty cell. Returns false if the board is full.
```

```
    bool findEmpty(int& r, int& c) const {
        for (r = 0; r < SIZE; ++r)
            for (c = 0; c < SIZE; ++c)
                if (cells[r][c] == 0) return true;
        return false;
    }

    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (cells[r][c] == 0) out << '_';
                else out << cells[r][c];
                out << ' ';
            }
            out << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> cells;
};

// Solves a Sudoku board with recursive depth-first search and backtracking,
// collecting every solution.
class SudokuSolver {
public:
    explicit SudokuSolver(const SudokuBoard& start) : board(start) {}

    const std::vector<SudokuBoard>& solveAll() {
        solutions.clear();
        search();
        return solutions;
    }

private:
    SudokuBoard board;
    std::vector<SudokuBoard> solutions;

    void search() {
        int r, c;
        if (!board.findEmpty(r, c)) {
            // All 81 cells are filled: record a solution.
            solutions.push_back(board);
            return;
        }
        std::vector<int> options = board.candidates(r, c);
        if (options.empty()) {
            return; // dead end, backtrack
        }
        for (int v : options) {
            board.set(r, c, v);
            search();
            board.set(r, c, 0); // undo
        }
    }
};

// Drives the program: loads each file, prints it, solves it, prints results.
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::vector<std::string>& files) : filenames(files) {}

    void run() const {
        for (const auto& name : filenames) {
            std::cout << "==============================\n";
            std::cout << "Puzzle file: " << name << "\n";
            std::cout << "==============================\n";

            SudokuBoard board;
            if (!board.loadFromFile(name)) {
                std::cout << "Could not read " << name << "\n\n";
                continue;
```

```
            }

            std::cout << "Puzzle:\n";
            board.print(std::cout);
            std::cout << '\n';

            SudokuSolver solver(board);
            const auto& solutions = solver.solveAll();

            if (solutions.empty()) {
                std::cout << "No solution found\n\n";
            } else {
                for (size_t i = 0; i < solutions.size(); ++i) {
                    std::cout << "Solution " << (i + 1) << ":\n";
                    solutions[i].print(std::cout);
                    std::cout << '\n';
                }
                std::cout << "Solutions found: " << solutions.size() << "\n\n";
            }
        }
    }

private:
    std::vector<std::string> filenames;
};

int main() {
    PuzzleRunner runner({"puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
                         "puzzle4.txt", "puzzle5.txt"});
    runner.run();
    return 0;
}
```

Appendix B – Raw Code Generated by Gemini

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    static const int GRID_SIZE = 9;
    static const int SUBGRID_SIZE = 3;

    int grid[GRID_SIZE][GRID_SIZE];
    int originalGrid[GRID_SIZE][GRID_SIZE];
    int solutionCount;

    // Finds the next empty cell (value 0). Returns true if found, false if grid is complete.
    bool findEmptyCell(int& row, int& col) const {
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                if (grid[r][c] == 0) {
                    row = r;
                    col = c;
                    return true;
                }
            }
        }
        return false;
    }

    // Computes list of candidate digits (1-9) not used in the cell's row, column, or 3x3 block.
    std::vector<int> getCandidates(int row, int col) const {
        std::vector<bool> used(10, false);

        // Check row
        for (int c = 0; c < GRID_SIZE; ++c) {
            if (grid[row][c] >= 1 && grid[row][c] <= 9) {
                used[grid[row][c]] = true;
            }
        }

        // Check column
        for (int r = 0; r < GRID_SIZE; ++r) {
            if (grid[r][col] >= 1 && grid[r][col] <= 9) {
                used[grid[r][col]] = true;
            }
        }

        // Check 3x3 block
        int startRow = (row / SUBGRID_SIZE) * SUBGRID_SIZE;
        int startCol = (col / SUBGRID_SIZE) * SUBGRID_SIZE;
        for (int r = 0; r < SUBGRID_SIZE; ++r) {
            for (int c = 0; c < SUBGRID_SIZE; ++c) {
                int val = grid[startRow + r][startCol + c];
                if (val >= 1 && val <= 9) {
                    used[val] = true;
                }
            }
        }

        // Collect available candidates
        std::vector<int> candidates;
        for (int num = 1; num <= 9; ++num) {
            if (!used[num]) {
                candidates.push_back(num);
            }
        }
        return candidates;
    }
```

```
    // Recursive depth-first search with backtracking to find all solutions.
    void solveDFS() {
        int row = -1, col = -1;

        // If no empty cell is found, all 81 cells are validly filled.
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printGrid(grid, false);
            std::cout << "\n";
            return;
        }

        std::vector<int> candidates = getCandidates(row, col);

        // If no candidates are available, backtrack.
        if (candidates.empty()) {
            return;
        }

        // Try each candidate recursively
        for (int candidate : candidates) {
            grid[row][col] = candidate;
            solveDFS();
            grid[row][col] = 0; // Backtrack
        }
    }

public:
    SudokuSolver() : solutionCount(0) {
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                grid[r][c] = 0;
                originalGrid[r][c] = 0;
            }
        }
    }

    // Loads a 9x9 grid from a text file where '_' represents an empty cell.
    bool loadFromFile(const std::string& filename) {
        std::ifstream inFile(filename);
        if (!inFile.is_open()) {
            std::cerr << "Error: Unable to open file " << filename << "\n";
            return false;
        }

        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                std::string token;
                if (!(inFile >> token)) {
                    std::cerr << "Error: File format invalid or incomplete in " << filename << "\n";
                    inFile.close();
                    return false;
                }

                if (token == "_") {
                    grid[r][c] = 0;
                    originalGrid[r][c] = 0;
                } else {
                    grid[r][c] = token[0] - '0';
                    originalGrid[r][c] = grid[r][c];
                }
            }
        }

        inFile.close();
        return true;
    }

    // Prints a 9x9 grid to stdout.
    void printGrid(const int g[GRID_SIZE][GRID_SIZE], bool showBlankAsUnderscore) const {
```

```
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                if (showBlankAsUnderscore && g[r][c] == 0) {
                    std::cout << "_";
                } else {
                    std::cout << g[r][c];
                }
                if (c < GRID_SIZE - 1) {
                    std::cout << " ";
                }
            }
            std::cout << "\n";
        }
    }

    // Processes a single puzzle file.
    void processPuzzle(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "========================================\n";

        if (!loadFromFile(filename)) {
            std::cout << "Failed to load puzzle.\n\n";
            return;
        }

        std::cout << "Original Puzzle:\n";
        printGrid(originalGrid, true);
        std::cout << "\n";

        solutionCount = 0;
        solveDFS();

        if (solutionCount == 0) {
            std::cout << "No solution found\n\n";
        }
    }

    // Reads and solves puzzle1.txt through puzzle5.txt.
    void runFivePuzzles() {
        const std::string files[5] = {
            "puzzle1.txt",
            "puzzle2.txt",
            "puzzle3.txt",
            "puzzle4.txt",
            "puzzle5.txt"
        };

        for (int i = 0; i < 5; ++i) {
            processPuzzle(files[i]);
        }
    }
};

int main() {
    SudokuSolver solver;
    solver.runFivePuzzles();
    return 0;
}
```